

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

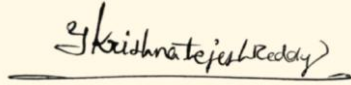
CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA1003	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Krishna Tejesh Reddy.Y	Reg. No.:	192411200
Date:	26-8-2026	Duration:	60 minutes

Code of Conduct

I _ Krishna Tejesh Reddy .Y(192411200) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



Annexure A – CI/CD Pipeline Design Exercise

CI/CD PIPELINE DESIGN FOR CAMPUS PLACEMENTS AND RECRUITMENT PORTAL

1. Introduction

The **Campus Placements and Recruitment Portal** is a web-based application that connects students, recruiters, and placement officers through a centralized recruitment platform. The system provides features such as student registration, profile management, resume upload, job posting, eligibility verification, job application, candidate shortlisting, interview scheduling, selection-result management, and placement reporting.

Since the portal handles continuously changing student, recruiter, and recruitment information, frequent software updates may be required. Manual integration, testing, and deployment can introduce errors and delays. Therefore, a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline is proposed to automate code integration, building, testing, quality analysis, packaging, and deployment.

The CI/CD pipeline ensures that every code change is automatically validated before being deployed. It improves software quality, reduces deployment errors, provides faster feedback to developers, and supports reliable delivery of new features.

This design follows the assessment requirement to identify CI/CD needs, design pipeline stages, select suitable tools, integrate automated testing and quality checks, and define deployment, security, and rollback strategies.

2. CI/CD Requirements Analysis

For the Campus Placements and Recruitment Portal, the CI/CD pipeline should satisfy the following requirements:

1. Developers should be able to maintain the source code using a version-control system.
2. Every code commit should automatically trigger the pipeline.
3. The application should be automatically built.
4. Automated unit and integration tests should be executed.
5. Code quality should be checked before deployment.
6. Security vulnerabilities should be detected early.

7. Successful builds should be packaged into deployable artifacts.
8. The application should first be deployed to a development or testing environment.
9. The application should be validated in the staging environment.
10. Production deployment should occur only after required quality gates are passed.
11. Failed deployments should be recoverable through rollback.
12. Pipeline results should be communicated to developers and administrators.
13. Sensitive information such as database credentials and API keys should not be stored directly in source code.
14. Application logs and deployment health should be monitored.
- 15.

3. Proposed CI/CD Architecture

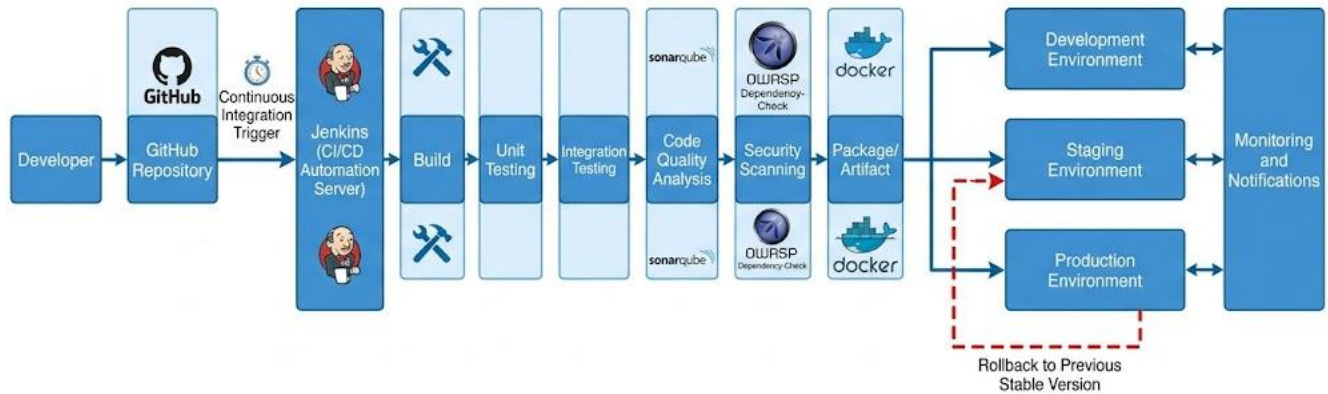
The proposed pipeline follows this workflow:

**Developer → Git Repository → Build → Automated Testing → Code Quality & Security → Package
→ Development → Staging → Production → Monitoring**

The major stages are:

1. Source Code Management
2. Build
3. Unit Testing
4. Integration Testing
5. Code Quality Analysis
6. Security Scanning
7. Packaging
8. Development Deployment
9. Staging Deployment
10. Production Deployment
11. Monitoring and Notification
12. Rollback

CI/CD Pipeline for Campus Placements and Recruitment Portal



4. Source Code Management

Tool: Git and GitHub

The source code of the Campus Placements and Recruitment Portal will be maintained using **Git** and hosted on **GitHub**.

Developers can create separate branches for new features and bug fixes. After completing a change, the developer pushes the code to the repository and creates a pull request.

A typical branching structure can be:

```
main
|
├── develop
|   ├── feature/student-registration
|   ├── feature/job-application
|   └── feature/recruiter-dashboard
|
└── hotfix/security-fix
```

The main branch contains stable production-ready code, while feature branches are used for individual development tasks.

Benefits:

- Version control
- Collaboration
- Change tracking
- Branch management
- Code review
- Easy rollback to previous versions

5. Continuous Integration Trigger

Whenever developers push code or create a pull request, the CI/CD pipeline is automatically triggered.

Example workflow:

Code Commit → GitHub Webhook → Jenkins Pipeline

Jenkins receives the notification and starts the pipeline automatically.

The pipeline should stop if an important stage fails. For example, if unit tests fail, the application should not proceed to production deployment.

6. Build Stage

The build stage prepares the Campus Placements and Recruitment Portal for testing and deployment.

Depending on the selected implementation, the frontend and backend can be built separately.

Frontend

For a React-based frontend:

```
npm install
npm run build
```

Backend

For a Python-based backend:

```
pip install -r requirements.txt
```

The build process verifies that the required dependencies are available and that the application can be compiled/built successfully.

Expected Result

If the build is successful, the pipeline proceeds to automated testing.

If the build fails, the pipeline stops and sends a failure notification.

7. Automated Testing Strategy

Automated testing is an important part of the pipeline because the assessment requires testing and quality checks to be integrated into the CI/CD process.

The proposed testing strategy contains multiple levels.

7.1 Unit Testing

Unit tests verify individual functions or components.

Examples:

- Student registration validation
- Login validation
- CGPA eligibility calculation
- Application-status update
- Resume validation
- Recruiter job-post validation

Example:

Input:

CGPA = 8.2

Backlogs = 0

Branch = CSE

Expected:

Eligible

7.2 Integration Testing

Integration testing verifies communication between different components.

Examples:

- Frontend ↔ Backend API
- Backend ↔ Database
- Login ↔ Authentication service
- Job application ↔ Eligibility module
- Recruiter dashboard ↔ Candidate database

7.3 API Testing

API tests verify whether backend services return correct responses.

Examples:

POST /register

POST /login

GET /jobs

POST /applications

GET /applications/{student_id}

POST /interviews

The pipeline executes these tests automatically after successful application build.

7.4 End-to-End Testing

End-to-end testing validates complete user workflows.

Example:

Student Login → View Job → Check Eligibility → Apply → Receive Application Confirmation

Another workflow:

Recruiter Login → Create Job → View Candidates → Shortlist Candidate → Schedule Interview

7.5 Regression Testing

Regression testing ensures that newly introduced changes do not break existing functionality.

For example, if the developer modifies the eligibility module, previously working login, registration, job search, and application features should continue to work.

8. Code Quality Analysis

Tool: SonarQube

SonarQube can be integrated into the pipeline to analyze source code quality.

It can help identify:

- Bugs
- Code smells
- Duplicated code
- Maintainability problems
- Security-related issues
- Poor coding practices

A quality gate can be configured so that code does not proceed to deployment when critical quality requirements are not satisfied.

Workflow:

Build → SonarQube Analysis → Quality Gate

If the quality gate passes, the pipeline continues.

If it fails, the pipeline stops.

9. Security Scanning

Security should be integrated directly into the CI/CD pipeline rather than being performed only after deployment.

Security checks can include:

Dependency Scanning

Check third-party libraries for known vulnerabilities.

Secret Detection

Ensure that passwords, API keys, database credentials, and tokens are not committed to the repository.

Container Scanning

If Docker containers are used, the container image should be scanned for known vulnerabilities before deployment.

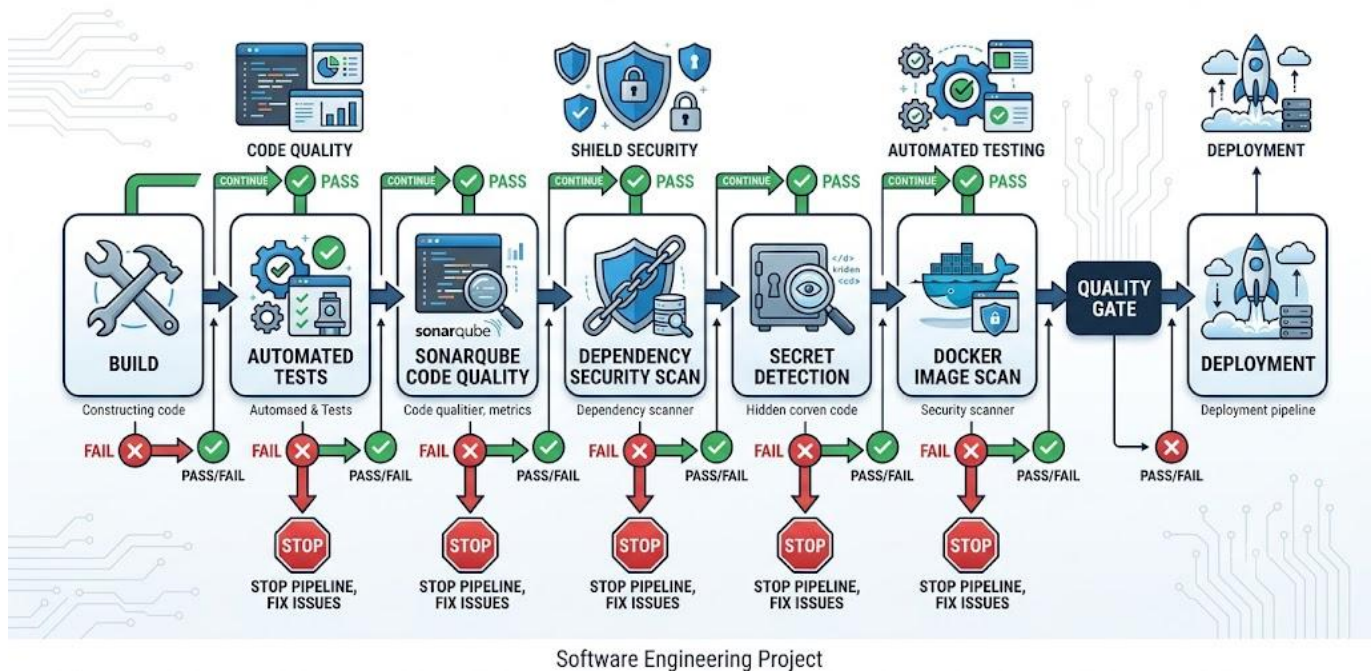
Authentication and Authorization Testing

Verify that:

- Students cannot access recruiter dashboards.
- Recruiters cannot access placement-officer functions.

- Unauthenticated users cannot access protected pages.
- Student information is protected from unauthorized access.

SECURITY AND QUALITY GATES IN CI/CD – CAMPUS RECRUITMENT PORTAL



10. Packaging

After successful testing and quality checks, the application is packaged into a deployable artifact.

Docker can be used to package the application and its dependencies into containers.

For example:

Campus Placement Portal



Docker Image



Container Registry



Deployment Environment

Containerization provides consistency between development, testing, and production environments.

It helps reduce the problem of:

“It works on my machine but does not work in production.”

11. Deployment Environments

The proposed system uses three major environments.

11.1 Development Environment

Used by developers for implementing and testing new features.

Example:

Feature Branch → Development Deployment

11.2 Staging Environment

Staging should closely resemble the production environment.

It is used for:

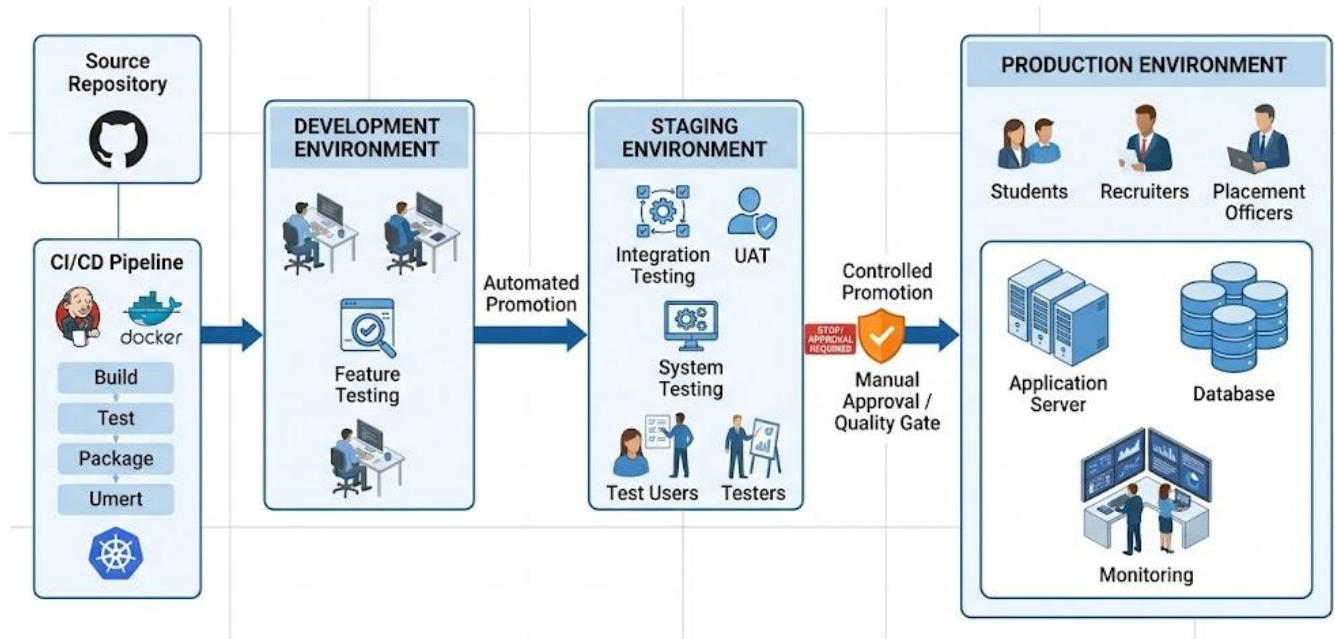
- Integration testing
- System testing
- User acceptance testing
- Final validation

11.3 Production Environment

Production is the live environment accessed by actual students, recruiters, and placement officers.

Only successfully tested and approved builds should be deployed to production.

CI/CD Deployment Environments – Campus Placements and Recruitment Portal



S

12. Production Deployment Strategy

A controlled deployment strategy should be used to reduce the risk of introducing faulty software into production.

The proposed deployment sequence is:

Code Commit



Build



Automated Testing



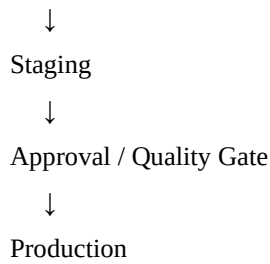
Quality & Security Checks



Package



Development



Production deployment should occur only when:

- Build succeeds
- Tests pass
- Quality gate passes
- Security checks pass
- Staging validation succeeds
- Required approval is obtained

13. Rollback Strategy

Rollback is required when a new production release introduces serious defects.

For example:

Suppose version **v2.0** is deployed and causes the job application module to fail.

The system can immediately return to the previous stable version:

v1.9 → Stable Production Version

v2.0 → Failed Release



Rollback



v1.9 → Restored Production

Rollback can be triggered when:

- Critical application errors occur.
- Database operations fail.

- Login stops working.
- Job applications cannot be submitted.
- Security vulnerabilities are discovered.
- Production health checks fail.

Previous Docker images or deployment artifacts should be retained so that a stable version can be restored quickly.

14. Monitoring and Notifications

After deployment, the application should be monitored continuously.

Monitoring should include:

- Application availability
- Server health
- API response time
- Error rates
- Database connectivity
- CPU and memory usage
- Failed login attempts
- Deployment status

Notifications

Developers or administrators should receive notifications when:

- Pipeline succeeds
- Pipeline fails
- Tests fail
- Security scan fails
- Production deployment succeeds
- Rollback occurs

Possible notification channels include email or team communication tools.

15. CI/CD Security Practices

Security should be incorporated throughout the pipeline.

15.1 Secure Credentials

Database passwords, API keys, and tokens should be stored using secure credentials management rather than directly in source code.

15.2 Access Control

Only authorized developers should be allowed to modify production-related configurations.

15.3 Branch Protection

The production branch should require code review and successful automated checks before merging.

15.4 Dependency Security

Third-party dependencies should be regularly scanned for vulnerabilities.

15.5 Secure Deployment

Production deployment credentials should be restricted to the CI/CD system and authorized administrators.

15.6 Data Protection

Student information such as academic records, contact details, resumes, and application information should be protected from unauthorized access.

16. CI/CD Pipeline Workflow

The complete workflow can be described as follows:

Step 1 – Developer Commit

Developer modifies the Campus Placement Portal code and pushes the changes to GitHub.

Step 2 – Pipeline Trigger

GitHub triggers Jenkins through a webhook.

Step 3 – Source Checkout

Jenkins retrieves the latest source code.

Step 4 – Build

The frontend and backend dependencies are installed and the application is built.

Step 5 – Unit Testing

Individual functions and components are tested automatically.

Step 6 – Integration Testing

Communication between frontend, backend, APIs, and database is tested.

Step 7 – Code Quality

SonarQube analyzes the code.

Step 8 – Security Scanning

Dependencies, secrets, and container images are scanned.

Step 9 – Packaging

The application is packaged into a Docker image.

Step 10 – Development Deployment

The application is deployed to the development environment.

Step 11 – Staging Deployment

The validated application is promoted to staging.

Step 12 – Quality Gate

The pipeline verifies that all required checks have passed.

Step 13 – Production Deployment

The approved version is deployed to production.

Step 14 – Monitoring

The production system is monitored for errors and performance issues.

Step 15 – Rollback

If critical problems occur, the previous stable version is restored.

17. Tools and Technologies

The following tools are suitable for the proposed CI/CD pipeline.

Pipeline Stage	Tool	Purpose
Source Control	Git	Version control
Repository	GitHub	Store and collaborate on source code
CI/CD Automation	Jenkins	Automate pipeline
Frontend Build	npm	Install dependencies/build frontend
Backend Build	Python/pip	Install backend dependencies
Unit Testing	Jest / PyTest	Automated unit testing
API Testing	Postman/Newman	Automated API validation

Code Quality	SonarQube	Static code analysis
Security	OWASP Dependency-Check	Dependency vulnerability scanning
Containerization	Docker	Package application
Registry	Docker Registry	Store container images
Deployment	Docker/Cloud Server	Deploy application
Monitoring	Prometheus/Grafana	Monitor system health
Notification	Email/Team Tool	Pipeline notifications

18. Justification for Tool Selection

Git and GitHub

Git provides distributed version control, while GitHub provides centralized repository hosting and collaboration.

Jenkins

Jenkins is suitable because it can automate multiple CI/CD stages and integrate with GitHub, testing tools, SonarQube, Docker, and deployment environments.

Docker

Docker provides consistent application environments and simplifies deployment.

SonarQube

SonarQube provides automated code-quality analysis and quality gates.

OWASP Dependency-Check

It helps identify vulnerable third-party dependencies before deployment.

PyTest/Jest

Automated testing frameworks can execute unit tests during the CI process.

Prometheus/Grafana

These tools can provide monitoring and visualization of application/system metrics.

19. Example Jenkins Pipeline

A simplified conceptual pipeline for the Campus Placement Portal is:

```
pipeline {  
  stages {  
  
    Stage 1: Checkout  
      ↓  
    Stage 2: Build  
      ↓  
    Stage 3: Unit Test  
      ↓  
    Stage 4: Integration Test  
      ↓  
    Stage 5: SonarQube Analysis  
      ↓  
    Stage 6: Security Scan  
      ↓  
    Stage 7: Docker Build  
      ↓  
    Stage 8: Deploy to Development  
      ↓  
    Stage 9: Deploy to Staging  
      ↓  
    Stage 10: Quality Gate  
      ↓  
    Stage 11: Deploy to Production  
      ↓  
    Stage 12: Monitor  
  }  
}
```

If any critical stage fails, the pipeline stops and the team is notified.

20. Failure Handling

Failure	Pipeline Action
Build failure	Stop pipeline and notify developer
Unit-test failure	Stop pipeline
Integration-test failure	Stop pipeline
Code-quality failure	Stop pipeline
Security vulnerability	Stop or block deployment
Docker build failure	Stop pipeline
Staging failure	Prevent production deployment
Production health-check failure	Trigger rollback
Deployment failure	Restore previous stable release

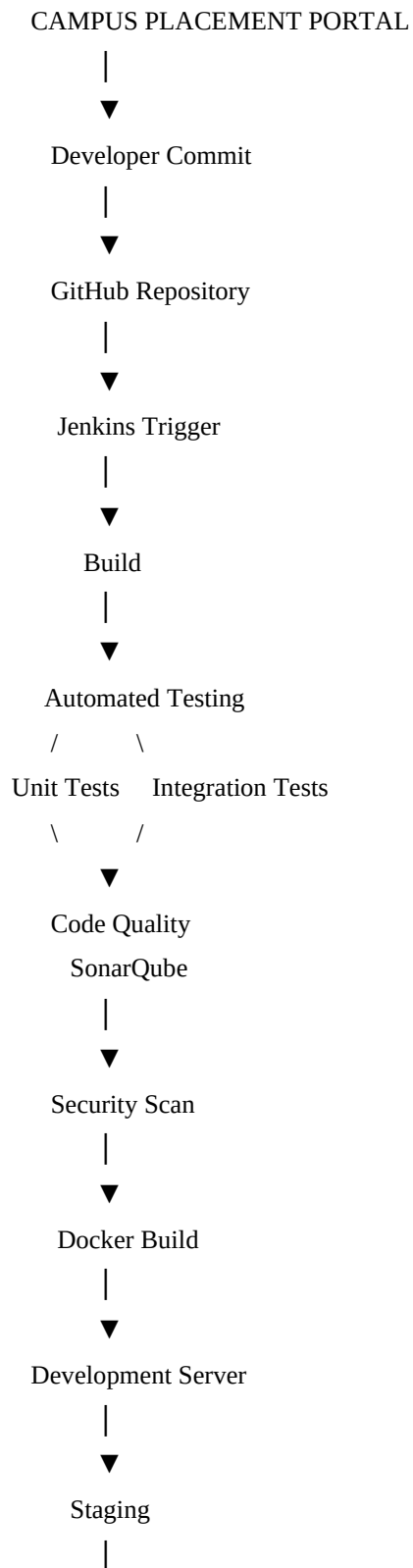
21. Benefits of the Proposed CI/CD Pipeline

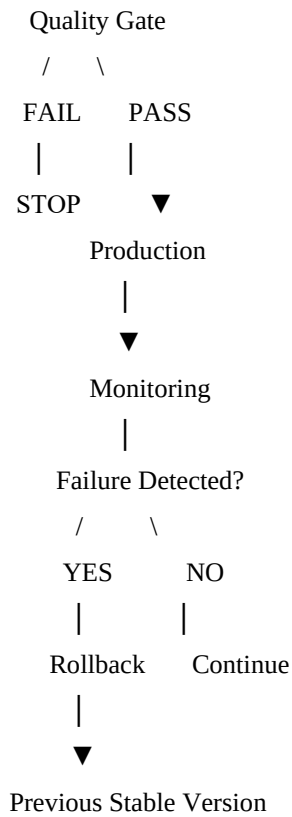
The proposed pipeline provides several benefits to the Campus Placements and Recruitment Portal:

1. Faster software delivery.
2. Automated testing after every major code change.
3. Early detection of defects.
4. Improved code quality.
5. Better security.
6. Consistent deployment environments.
7. Reduced manual deployment errors.
8. Faster rollback after failed deployments.
9. Improved collaboration among developers.
10. Continuous monitoring of the production system.
11. Better reliability during high-demand placement periods.
12. Reduced risk when introducing new features.

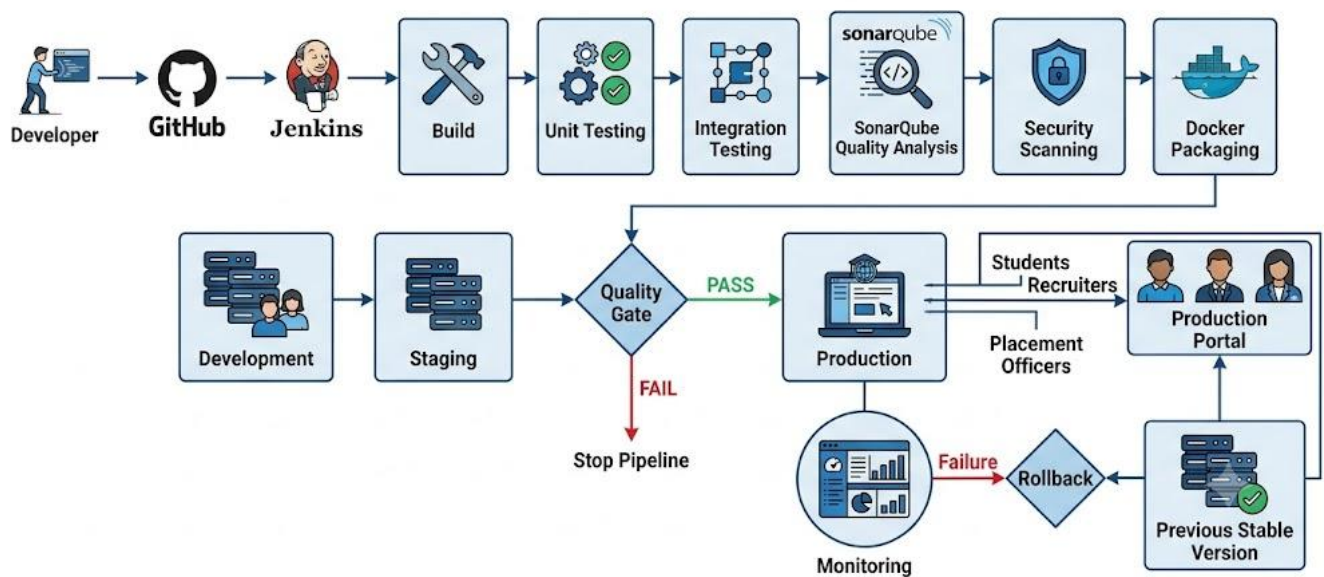
22. Overall CI/CD Flow

The complete proposed workflow can be summarized as:





Campus Placements and Recruitment Portal – Complete CI/CD Workflow



23. Conclusion

The proposed CI/CD pipeline for the **Campus Placements and Recruitment Portal** provides an automated and systematic approach for software development, testing, quality assurance, security, and deployment.

The pipeline integrates source-code management, automated builds, unit testing, integration testing, code-quality analysis, security scanning, Docker packaging, development and staging environments, controlled production deployment, monitoring, notifications, and rollback.

By automating these activities, the system can deliver reliable software updates more quickly while reducing manual errors. Security and quality checks are performed before production deployment, and the rollback mechanism provides protection against failed releases.

Therefore, the proposed CI/CD architecture supports the CO4 objective of implementing **robust testing, quality assurance, DevOps practices, security, reliability, and ethical handling of software systems.**

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured

Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25